

```

1  (*
2                                     CS 51 Lab 19
3                                     Gray-Scott Two-Variable Reaction-Diffusion CA
4                                     https://en.wikipedia.org/wiki/Gray%E2%80%93Scott\_model
5
6  This CA uses a *pair* of floats as its cell state, demonstrating that
7  the Automaton functor works for any state type -- including multi-
8  variable ones.  The two variables are:
9
10     u  -- "food" concentration (continuously replenished)
11     v  -- "activator" concentration (self-catalytic, decays)
12
13  The governing equations (Euler-integrated, dt = 1):
14
15     U' = U + Du *  $\nabla^2 U$  - U·V2 + F·(1 - U)
16     V' = V + Dv *  $\nabla^2 V$  + U·V2 - (F+k)·V
17
18  From equations to code: the time derivatives are discretized with
19  Euler's method at dt = 1, so each right-hand-side term is simply
20  added to the current value.  The spatial Laplacian  $\nabla^2 U$  -- which in
21  the continuous case measures how much a value differs from its local
22  neighborhood average -- is approximated by a weighted sum over the
23  8-cell Moore neighborhood minus the center cell.  Edge neighbors
24  (sharing a side) receive weight +0.2, corner neighbors weight +0.05,
25  and the center -1; the weights sum to zero, so a spatially uniform
26  field correctly yields  $\nabla^2 U = 0$ .  This isotropic 9-point stencil is a
27  standard finite-difference approximation of the 2D Laplacian; see
28  https://en.wikipedia.org/wiki/Discrete\_Laplace\_operator for details.
29  The diffusion coefficients Du = 0.2 and Dv = 0.1 are small enough to
30  keep the Euler integration stable at this step size.
31
32  Five named presets are defined below; set cPRESET to choose one.
33  Each preset bundles F, k, and the appropriate initial conditions so
34  the three values cannot be set inconsistently.
35  *)
36
37  module G = Graphics ;;
38
39  (* Automaton parameters *)
40  let cGRID_SIZE = 100 ;;
41
42  (* Diffusion coefficients -- fixed for all presets *)
43  let cDU = 0.2      (* diffusion rate of U (food) *)
44  let cDV = 0.1      (* diffusion rate of V (activator) *)
45
46  (* seeding -- `Patches` places five 7x7 seed squares of (0.5, 0.25)
47  into an otherwise empty background; patterns grow outward from each
48  seed.  `Uniform (u0, v0)` fills every cell near the Turing-unstable
49  homogeneous steady state (u0, v0) with small noise; patterns nucleate
50  everywhere simultaneously.  The steady state satisfies u0.v0 = F+k
51  and u0.v0^2 = F.(1-u0). *)
52  type seeding =

```

```

53 | Patches
54 | Uniform of float * float
55
56 (* preset -- Bundles F, k, and the matching initial conditions so the
57    three cannot be set inconsistently. Set cPRESET to one of the five
58    named values below; that is the only line you need to change. *)
59 type preset = {
60     name      : string;
61     f         : float;
62     k         : float;
63     seeding   : seeding;
64 }
65
66 let spots      = { name = "spots";      f = 0.035; k = 0.065; seeding = Patches }
67 let labyrinths = { name = "labyrinths"; f = 0.040; k = 0.060; seeding = Uniform (0.50, 0.20) }
68 let worms      = { name = "worms";      f = 0.025; k = 0.050; seeding = Patches }
69 let holes      = { name = "holes";      f = 0.039; k = 0.058; seeding = Uniform (0.60, 0.10) }
70 let solitons   = { name = "solitons";    f = 0.014; k = 0.054; seeding = Patches }
71
72 let cPRESET = solitons
73
74 let cF = cPRESET.f
75 let cK = cPRESET.k
76
77 (* Rendering parameters *)
78 let cCOLOR_LEGEND = G.rgb 200 200 100 ;;
79 let cSIDE = 8 ;;
80 let cRENDER_FREQUENCY = 4 ;; (* render every 4 ticks; patterns form slowly *)
81 let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"
82
83 (* State is a pair (u, v): food and activator concentrations *)
84 type gs_state = float * float
85
86 (* offset index off -- Returns `index` offset by `off` with wraparound *)
87 let offset (index : int) (off : int) : int =
88     ((index + off) mod cGRID_SIZE + cGRID_SIZE) mod cGRID_SIZE ;;
89
90 (* gs_update grid i j -- Returns the updated (u, v) for the cell at
91    position i, j using the Gray-Scott reaction-diffusion equations. *)
92 let gs_update (grid : gs_state array array) (i : int) (j : int) : gs_state =
93     let (u, v) = grid.(i).(j) in
94     (* 9-point Laplacian: edge neighbors have weight 0.2, corners 0.05 *)
95     let lu = ref (~-. u) and lv = ref (~-. v) in
96     for di = ~-1 to 1 do
97         for dj = ~-1 to 1 do
98             if di <> 0 || dj <> 0 then begin
99                 let w = if di = 0 || dj = 0 then 0.2 else 0.05 in
100                 let ni = offset i di and nj = offset j dj in
101                 let (nu, nv) = grid.(ni).(nj) in
102                 lu := !lu +. w *. nu;
103                 lv := !lv +. w *. nv
104             end
105         done
106     done;

```

```

107 let uvv = u *. v *. v in
108 let u' = u +. cDU *. !lu -. uvv +. cF *. (1. -. u) in
109 let v' = v +. cDV *. !lv +. uvv -. (cF +. cK) *. v in
110 (max 0. (min 1. u'), max 0. (min 1. v')) ;;
111
112 (* gs_initialize grid -- Seeds `grid` according to cPRESET.seeding:
113    Patches places five 7x7 squares of (0.5, 0.25) into an otherwise
114    empty grid; Uniform fills every cell near the steady state (u0, v0)
115    with small noise. *)
116 let gs_initialize (grid : gs_state array array) : unit =
117   match cPRESET.seeding with
118   | Patches ->
119     for _ = 1 to 5 do
120       let ci = 10 + Random.int (cGRID_SIZE - 20) in
121       let cj = 10 + Random.int (cGRID_SIZE - 20) in
122       for di = ~-3 to 3 do
123         for dj = ~-3 to 3 do
124           grid.(ci + di).(cj + dj) <- (0.5, 0.25)
125         done
126       done
127     done
128   | Uniform (u0, v0) ->
129     for i = 0 to cGRID_SIZE - 1 do
130       for j = 0 to cGRID_SIZE - 1 do
131         let u_noise = Random.float 0.1 -. 0.05 in
132         let v_noise = Random.float 0.1 -. 0.05 in
133         grid.(i).(j) <- (u0 +. u_noise, v0 +. v_noise)
134       done
135     done ;;
136
137 module GSSpec : (Cellular.AUT_SPEC with type state = gs_state) =
138   struct
139     type state = gs_state
140     let name = "Gray-Scott: " ^ cPRESET.name
141     let initial : state = (1., 0.)
142     let grid_size = cGRID_SIZE
143     let initialize = gs_initialize
144     let update = gs_update
145     (* Warm-to-cool false color: U (food) drives orange-yellow,
146        V (activator, scaled x5) shifts toward deep blue-purple.
147        Background (U≈1, V≈0) appears bright orange-yellow;
148        active stripes (U≈0.4, V≈0.2) appear deep blue-purple. *)
149     let cell_color ((u, v) : state) : G.color =
150       let v' = min 1. (v *. 5.) in
151       G.rgb (int_of_float (u *. (1. -. v' *. 0.6) *. 255. +. v' *. 40.))
152             (int_of_float (u *. (1. -. v') *. 180. +. v' *. 40.))
153             (int_of_float (v' *. 160.))
154     let side_size = cSIDE
155     let legend_color = cCOLOR_LEGEND
156     let font = cFONT
157     let render_frequency = cRENDER_FREQUENCY
158   end ;;
159
160 module Aut = Cellular.Automaton (GSSpec) ;;

```

```

161
162 (* main seed -- Initializes the grid according to cPRESET.seeding and runs
163    the Gray-Scott simulation. *)
164 let main seed =
165     Random.init seed;
166     Aut.graphics_init ();
167     Aut.run_grid (Aut.initialized_grid ()) ;;
168
169 let _ =
170     if Array.length Sys.argv <= 1 then
171         Printf.printf "Usage: %s <seed>\n  where <seed> is integer seed\n"
172             Sys.argv.(0)
173     else
174         main (int_of_string Sys.argv.(1)) ;;

```